1. **Exist ticket**

SOLUTION

- **The expression results in 81 because of integer division.**
- When dividing whole numbers in code, the fractional part is truncated rather than rounded.
- **To get 81.67, you must use floating-point division.**
- This is achieved by changing the divisor to a decimal, like `(70 + 85 + 90) / 3.0`.

Data-Type Selection Table

Variable Data Context	Best C++ Data Type	Bit Width	Reasons for Choice
Monetary Values / Wages (e.g., R81.67)	`double`	64-bit	Handles decimal values with high precision required for financial amounts.
Item Quantities / Stock Counts	`int`	32-bit	Quantities are discrete whole numbers; negative counts are theoretically invalid but `int` allows error-checking.
Averages / Calculations	`double` or `float`	64/32-bit	Prevents integer division truncation and preserves fractional math accuracy.
Product Codes / ID Numbers (Fixed Format)	`std::string`	Variable	Keeps leading zeros intact (e.g., "0012") and allows alphanumeric combinations.
Product Names / Descriptions	`std::string`	Variable	Stores dynamic text sequences of variable lengths.
Status Flags (e.g., IsTaxable, IsPaid)	`bool`	8-bit	Perfect for binary state evaluation (either `true` or `false`).
Menu Selection Characters (e.g., 'A', 'B	`char`	8-bit	Highly efficient for storing individual single-character input inputs.

2. PRODUCT AND WAGES WITH EXTENSIONS

```
#include <iostream>
#include <iomanip>

int main() {
    std::cout << std::fixed << std::setprecision(2);

    // --- 1. PRODUCT SECTION (Fixes Integer Division using 3.0) ---
    double prod1 = 70, prod2 = 85, prod3 = 90;
    double productAverage = (prod1 + prod2 + prod3) / 3.0;

    std::cout << "--- Product Summary ---\n";
    std::cout << "Total Product Cost: R" << (prod1 + prod2 + prod3) << "\n";
    std::cout << "Average Product Price: R" << productAverage << "\n\n";

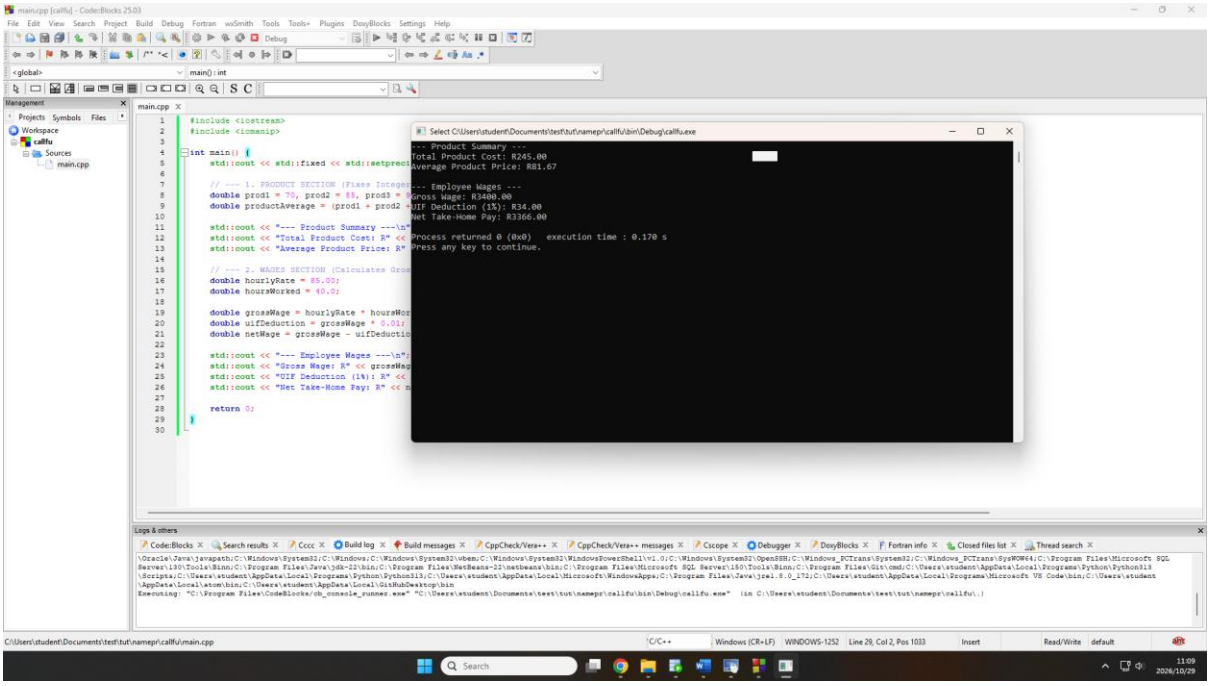
    // --- 2. WAGES SECTION (Calculates Gross & Net with 1% UIF) ---
    double hourlyRate = 85.00;
    double hoursWorked = 40.0;

    double grossWage = hourlyRate * hoursWorked;
    double uifDeduction = grossWage * 0.01; // 1% UIF
    double netWage = grossWage - uifDeduction;

    std::cout << "--- Employee Wages ---\n";
    std::cout << "Gross Wage: R" << grossWage << "\n";
    std::cout << "UIF Deduction (1%): R" << uifDeduction << "\n";
    std::cout << "Net Take-Home Pay: R" << netWage << "\n";

    return 0;
}
```

2. PRODUCT AND WAGES WITH EXTENSIONS



3. Calculation drill with corrections

Mathematical Goal	Broken C++ Expression	Actual Output	Corrected C++ Expression	Strategic Reason for Correction
Average of 3 Marks Calculate average for 70, 85, and 90.	`(70 + 85 + 90) / 3`	**81**	`(70 + 85 + 90) / 3.0`	Changing the divisor to `3.0` forces **floating-point division**, preserving the decimal remainder (**81.67**).
Add 15% VAT Apply tax to a subtotal of R200.	`200 * (15 / 100)`	**0.00**	`200 * (15.0 / 100.0)`	Integer division `15 / 100` truncates down to `0`. Forcing decimals keeps the calculation ratio at `0.15`.
Percentage Grade Scoring 18 out of 25.	`(18 / 25) * 100`	**0.00**	`(static_cast<double>(18) / 25) * 100`	Explicitly casting the numerator to a `double` ensures the fractional split evaluated inside brackets isn't lost to integer truncation.
Split Bill by Half Halving a	`155 / 2`	**77**	`155 / 2.0`	Dividing two integer types drops the half-rand completely. The decimal literal `2.0`

balance of R155.

accurately yields **77.50**.

4 . RECEIPT AND GETLINE

```
#include <iostream>
#include <iomanip>
#include <string>
#include <limits> // Required for std::numeric_limits

int main() {
   // --- PART 1: GETLINE FIX (Buffer Management) ---
   int studentNumber;
   std::string studentName;

   std::cout << "========================================\n";
   std::cout << "        CAMPUS PROFILE REGISTRATION    \n";
   std::cout << "========================================\n";

   std::cout << "Enter TUT Student Number: ";
   std::cin >> studentNumber;

   // FIX: Flushes the hidden newline character '\n' out of the input stream
   std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

   std::cout << "Enter Full Name and Surname: ";
   std::getline(std::cin, studentName); // Safely captures spaces without skipping

   // --- PART 2: RECEIPT GENERATION (Formatted Checkout Layout) ---
   std::string item1 = "Textbook", item2 = "Lab Coat", item3 = "Scientific Calc";
   double price1 = 350.00, price2 = 275.50, price3 = 199.90;
```

```
    double subtotal = price1 + price2 + price3;
    double vatAmount = subtotal * 0.15; // Standard 15% VAT
    double grandTotal = subtotal + vatAmount;

    // Printing Aligned Receipt Output
    std::cout << "\n========================================\n";
    std::cout << "        TUT CAMPUS BOOKSTORE          \n";
    std::cout << "========================================\n";
    std::cout << "Account: " << studentNumber << "\n";
    std::cout << "Buyer  : " << studentName << "\n";
    std::cout << "----------------------------------------\n";

    // Header alignment structure
    std::cout << std::left << std::setw(25) << "Item Description"
          << std::right << std::setw(15) << "Price (R)" << "\n";
    std::cout << "----------------------------------------\n";

    // Item line printing explicitly scaled to 2 decimal places
    std::cout << std::fixed << std::setprecision(2);
    std::cout << std::left << std::setw(25) << item1 << std::right << std::setw(15) << price1 << "\n";
    std::cout << std::left << std::setw(25) << item2 << std::right << std::setw(15) << price2 << "\n";
    std::cout << std::left << std::setw(25) << item3 << std::right << std::setw(15) << price3 << "\n";

    std::cout << "----------------------------------------\n";
    std::cout << std::left << std::setw(25) << "Subtotal:" << std::right << std::setw(15) << subtotal <<
"\n";
    std::cout << std::left << std::setw(25) << "VAT (15%):" << std::right << std::setw(15) << vatAmount
<< "\n";
    std::cout << "========================================\n";
    std::cout << std::left << std::setw(25) << "GRAND TOTAL:" << std::right << std::setw(15) <<
grandTotal << "\n";
    std::cout << "========================================\n";
```

```
    return 0;
}
```

5. **Debugging table for debug**

Error Class	Broken C++ Syntax Line	Symptoms / Compiler Error	Corrected C++ Syntax Line	Strategic Fix / Reason
Stream Bug	`cin >> studentNumber; getline(cin, name);`	Skips name entry entirely by reading an empty string.	`cin >> studentNumber; cin.ignore(INT_MAX, '\n'); getline(cin, name);`	Flushes the leftover newline character (`\n`) out of the input buffer before reading the text line.
Logic Truncation	`double avg = (m1+m2+m3) / 3;`	Strips decimals completely (e.g., returns `81.00` instead of `81.67`).	`double avg = (m1+m2+m3) / 3.0;`	Forcing a floating-point literal (`3.0`) elevates expression evaluation rules to float math.
Type Mismatch	`int wages = 85.50 * hours;`	Dropped decimal data warning; trims away minor currency change.	`double wages = 85.50 * hours;`	Changes data container architecture to handle high-precision monetary data variants securely.
Scope Missing	`cout << "Enter Item Name";`	`'cout' was not declared in this scope` error flag raised.	`std::cout << "Enter Item Name";` *(or add `using namespace std;`)*	Attaches code namespace identifiers cleanly to direct output pipelines.
String Literal	`string message = 'Success';`	Invalid character constant compilation crash.	`string message = "Success";`	Uses double quotes (`" "`) required for dynamic strings instead of single quotes (`' '`) meant for single characters.
String Missing	`string employeeName;`	`'string' does not name a type` error code.	`#include <string> std::string employeeName;`	Includes the standard library template header before initializing variable labels.